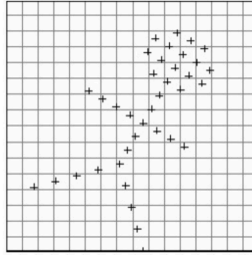
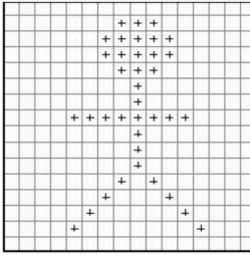


Bilinear_Interpolation

1. Motivation: When an image undergoes an affine transformation such as a rotation or scaling, the pixels in the image get moved around. Imagine a grayscale image (for simplicity) as a grid with the center of each grid having a float value. Most often the transformed coordinates (meshgrid) would be have decimal values and would not a corresponding pixel value in the original image.



- 2.
3. So for example, suppose that after rotating an image, we need to find the pixel value at the location (6.7, 3.2). The problem with this is that there is no such thing as fractional pixel locations.
4. Image processing affine transformation usually follows the 3-step pipeline:
 1. Grid generator: first we create a sampling grid composed of coordinates. For example, given a grayscale image, we create a meshgrid of same dimension, that is, evenly spaced and .
 2. We then apply the transformation matrix to the sampling grid generated in the step above.
 3. Grid sampler/interpolator (**bilinear interpolation**, etc): finally, we sample the resulting grid from the original image using the desired interpolation technique.
 4. In summary, to apply affine transformation to an image we do not apply the transformation to the image directly i.e. an apply transformation to an image of shape (H, W, C) instead, what we do is create a tensor containing image coordinates (meshgrid) of shape (H, W, 2). Then we apply transformation to the meshgrid coordinates. Finally, perform sampling or interpolation from the original image to get the pixel value for each grid cell in the transformed meshgrid. For example in grid cell with index $x=2$, $y=3$ we have (6.7, 2.3) is telling we have to sample from the coordinate $x=6.7$ and $y=2.3$ in the original image.
 5. Note: we must make sure that no value goes beyond the original image boundaries (use np.clip).
 6. Related: Spatial Transformer Networks.
5. Code:

```
# -*- coding: utf-8 -*-
"""
Created on Mon Jan 19 12:54:02 2026

@author: rsllim
"""

import cv2
import math
import numpy as np
import torch
import torch.nn.functional as F

def linear_resize(in_array, size):
    """
    Parameters
    -----
    in_array : input 1D array.
    size : desired size

    Returns
    -----
    resized array

    """
    ratio = (len(in_array) - 1) / (size - 1)
    out_array = []

    for i in range(size):
        low = math.floor(ratio * i)
        high = math.ceil(ratio * i)
        weight = ratio * i - low

        a = in_array[low]
        b = in_array[high]

        out_array.append(a * (1 - weight) + b * weight)
```

```

return np.array(out_array)

def bilinear_resize(image, height, width):
    """
    Parameters
    -----
    image : 2D numpy array.
    height : desired height
    width : desired width

    Returns
    -----
    resized image

    """
    img_height, img_width, img_channels = image.shape

    resized = np.zeros((height, width, img_channels))

    x_ratio = float(img_width - 1) / (width - 1) if width > 1 else 0
    y_ratio = float(img_height - 1) / (height - 1) if height > 1 else 0

    for i in range(height):
        for j in range(width):
            x_l, y_l = math.floor(x_ratio * j), math.floor(y_ratio * i)
            x_h, y_h = math.ceil(x_ratio * j), math.ceil(y_ratio * i)

            x_weight = (x_ratio * j) - x_l
            y_weight = (y_ratio * i) - y_l

            a = image[y_l, x_l]
            b = image[y_l, x_h]
            c = image[y_h, x_l]
            d = image[y_h, x_h]

            pixel = a * (1 - x_weight) * (1 - y_weight) \
                + b * x_weight * (1 - y_weight) \
                + c * y_weight * (1 - x_weight) \
                + d * x_weight * y_weight

            resized[i, j] = pixel

    return resized.astype(np.uint8)

def bilinear_resize_vectorized(image, height, width):
    """
    Parameters
    -----
    image : 2D numpy array.
    height : desired height
    width : desired width

    Returns
    -----
    resized image

    """
    img_height, img_width, img_channels = image.shape
    resized_img = []

    print(img_height)
    print(img_width)
    for c in range(img_channels):
        img_c = image[:, :, c].ravel()

        x_ratio = float(img_width - 1) / (width - 1) if width > 1 else 0
        y_ratio = float(img_height - 1) / (height - 1) if height > 1 else 0

        y, x = np.divmod(np.arange(height * width), width)

        x_l = np.floor(x_ratio * x).astype(np.uint32)
        y_l = np.floor(y_ratio * y).astype(np.uint32)

        x_h = np.ceil(x_ratio * x).astype(np.uint32)
        y_h = np.ceil(y_ratio * y).astype(np.uint32)

        x_weight = (x_ratio * x) - x_l
        y_weight = (y_ratio * y) - y_l

        a = img_c[y_l * img_width + x_l]
        b = img_c[y_l * img_width + x_h]
        c = img_c[y_h * img_width + x_l]
        d = img_c[y_h * img_width + x_h]

        resized = a * (1 - x_weight) * (1 - y_weight) + \
            b * x_weight * (1 - y_weight) + \
            c * y_weight * (1 - x_weight) + \
            d * x_weight * y_weight
        resized_img.append(resized.reshape(height, width).astype(np.uint8))

```

```

return np.stack(resized_img, axis=2)

def bilinear_resize_vectorized_backward(image, height, width, grad):
    """
    `image` is a 2-D array, holding the input image
    `height` and `width` are the desired spatial dimension of the new 2-D array.
    `grad` is a 2-D array of shape [height, width], holding the gradient to be
    backproped to `image`.
    """
    # This backprop implementation only works for a single channel
    img_height, img_width, _ = image.shape
    image = image.ravel()
    x_ratio = float(img_width - 1) / (width - 1) if width > 1 else 0
    y_ratio = float(img_height - 1) / (height - 1) if height > 1 else 0

    y, x = np.divmod(np.arange(height * width), width)

    x_l = np.floor(x_ratio * x).astype('int32')
    y_l = np.floor(y_ratio * y).astype('int32')

    x_h = np.ceil(x_ratio * x).astype('int32')
    y_h = np.ceil(y_ratio * y).astype('int32')

    x_weight = (x_ratio * x) - x_l
    y_weight = (y_ratio * y) - y_l

    grad = grad.ravel()
    # gradient wrt `a`, `b`, `c`, `d`
    d_a = (1 - x_weight) * (1 - y_weight) * grad
    d_b = x_weight * (1 - y_weight) * grad
    d_c = y_weight * (1 - x_weight) * grad
    d_d = x_weight * y_weight * grad
    # [4 * height * width]
    grad = np.concatenate([d_a, d_b, d_c, d_d])
    # [4 * height * width]
    indices = np.concatenate([y_l * img_width + x_l,
                              y_l * img_width + x_h,
                              y_h * img_width + x_l,
                              y_h * img_width + x_h])
    # we must route gradients in `grad` to the correct indices of `image` in
    # `indices`, e.g. only entries of indices `y_l * img_width + x_l` in `image`
    # gets the gradient backproped from `a`.
    # use numpy's broadcasting rule to generate 2-D array of shape
    # [4 * height * width, img_height * img_width]
    indices = (indices.reshape((-1, 1)) ==
               np.arange(img_height * img_width).reshape((1, -1)))
    d_image = np.apply_along_axis(lambda col: grad[col].sum(), 0, indices)
    return d_image.reshape((img_height, img_width))

if __name__ == '__main__':
    """ 1D Example """
    in_array = [1,2,3,4]
    out_array_smaller = linear_resize(in_array, 2)
    print(out_array_smaller)
    out_array_larger = linear_resize(in_array, 7)
    print(out_array_larger)

    """ 2D Example """
    # Load image
    img=cv2.imread('bender.png')
    print("Original image size", img.shape)
    cv2.imshow('image', img)
    cv2.waitKey(0)
    cv2.destroyAllWindows()

    # New image size
    height = 128
    width = 400
    # Bilinear resize without vectorize
    resized_img = bilinear_resize(img, height, width)
    print("resized_img.shape", resized_img.shape)
    cv2.imshow('Resized image', resized_img)
    cv2.waitKey(0)
    cv2.destroyAllWindows()

    # Using vectorized implementation (faster)
    resized_img = bilinear_resize_vectorized(img, height, width)
    print("resized_img.shape", resized_img.shape)
    cv2.imshow('Resized image', resized_img)
    cv2.waitKey(0)
    cv2.destroyAllWindows()

    """ Test bilinear resize vectorized backward """
    # Create pseudo tensor
    input_tensor = torch.randn(1, 3, 5, 8) # (N, C, H, W)

    ### Pytorch implementation
    input_tensor.requires_grad_(True)
    # Forward pass

```

```

output_tensor = F.interpolate(input_tensor, size=(6, 9), align_corners=True, mode='bilinear')
# Backward pass
# output_tensor.sum().backward()
output_tensor.sum().backward()
torch_grad = input_tensor.grad
print("torch_grad", torch_grad)
print("torch_grad.shape", torch_grad.shape)

### Manual implementation
print("")
# Convert (N, C, H, W) to (H, W, C) since our manual backprop only works for a single image
print("input_tensor.squeeze(0, 1)[: , np.newaxis].shape", input_tensor.squeeze(0).permute(1, 2, 0).shape)
grad = torch.ones((6, 9)) # Pytorch does this internally during backward
manual_grad = bilinear_resize_vectorized_backward(input_tensor.squeeze(0).permute(1, 2, 0).detach().cpu().numpy(),
6, 9, grad.detach().cpu().numpy())

print("manual_grad", manual_grad)
print("manual_grad.shape", manual_grad.shape)

```

Output:

```

[1. 4.]
[1. 1.5 2. 2.5 3. 3.5 4. ]
Original image size (338, 600, 3)
resized_img.shape (128, 400, 3)
338
600
resized_img.shape (128, 400, 3)
torch_grad tensor([[[[1.3500, 1.3500, 1.3500, 1.3500, 1.3500, 1.3500, 1.3500, 1.3500],
[1.3500, 1.3500, 1.3500, 1.3500, 1.3500, 1.3500, 1.3500, 1.3500],
[1.3500, 1.3500, 1.3500, 1.3500, 1.3500, 1.3500, 1.3500, 1.3500],
[1.3500, 1.3500, 1.3500, 1.3500, 1.3500, 1.3500, 1.3500, 1.3500],
[1.3500, 1.3500, 1.3500, 1.3500, 1.3500, 1.3500, 1.3500, 1.3500]],

[[1.3500, 1.3500, 1.3500, 1.3500, 1.3500, 1.3500, 1.3500, 1.3500],
[1.3500, 1.3500, 1.3500, 1.3500, 1.3500, 1.3500, 1.3500, 1.3500],
[1.3500, 1.3500, 1.3500, 1.3500, 1.3500, 1.3500, 1.3500, 1.3500],
[1.3500, 1.3500, 1.3500, 1.3500, 1.3500, 1.3500, 1.3500, 1.3500],
[1.3500, 1.3500, 1.3500, 1.3500, 1.3500, 1.3500, 1.3500, 1.3500]],

[[1.3500, 1.3500, 1.3500, 1.3500, 1.3500, 1.3500, 1.3500, 1.3500],
[1.3500, 1.3500, 1.3500, 1.3500, 1.3500, 1.3500, 1.3500, 1.3500],
[1.3500, 1.3500, 1.3500, 1.3500, 1.3500, 1.3500, 1.3500, 1.3500],
[1.3500, 1.3500, 1.3500, 1.3500, 1.3500, 1.3500, 1.3500, 1.3500],
[1.3500, 1.3500, 1.3500, 1.3500, 1.3500, 1.3500, 1.3500, 1.3500]]]])
torch_grad.shape torch.Size([1, 3, 5, 8])

input_tensor.squeeze(0, 1)[: , np.newaxis].shape torch.Size([5, 8, 3])
manual_grad [[1.35 1.35 1.35 1.35 1.35 1.35 1.35 1.35 1.35 1.35]
[1.35 1.35 1.35 1.35 1.35 1.35 1.35 1.35 1.35]
[1.35 1.35 1.35 1.35 1.35 1.35 1.35 1.35 1.35]
[1.35 1.35 1.35 1.35 1.35 1.35 1.35 1.35 1.35]
[1.35 1.35 1.35 1.35 1.35 1.35 1.35 1.35 1.35]]
manual_grad.shape (5, 8)

```

Affine Transformation

1. Code:

```

# -*- coding: utf-8 -*-
"""
Created on Tue Jul 14 13:13:31 2026

@author: reina
"""

import cv2
import numpy as np
import matplotlib.pyplot as plt

def show_image(title, img):
    """ Utility function to display an image. """
    img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    plt.figure(figsize=(6,6))
    plt.imshow(img_rgb)
    plt.title(title)
    plt.axis('off')
    plt.show()

img = cv2.imread('apple.jpg')

if img is None:
    print("Could not load OpenCV sample image.")
else:

```

```

show_image("Original Image", img)

def translate_image(img, tx, ty):
    """ Applies translation to an image."""
    T = np.array([[1, 0, tx],
                  [0, 1, ty]], dtype=np.float32)
    return cv2.warpAffine(img, T, (img.shape[1], img.shape[0]))

def scale_image_warp(img, sx, sy):
    """Applies scaling using warpAffine"""
    T = np.array([[sx, 0, 0],
                  [0, sy, 0]], dtype=np.float32)
    new_size = (int(img.shape[1] * sx), int(img.shape[0] * sy)) # Adjusted output size
    return cv2.warpAffine(img, T, new_size)

def rotate_image(img, angle):
    """Applies rotation to an image."""
    angle = np.deg2rad(angle)
    R = np.array([[np.cos(angle), -np.sin(angle), 0],
                  [np.sin(angle), np.cos(angle), 0]], dtype=np.float32)
    return cv2.warpAffine(img, R, (img.shape[1], img.shape[0]))

def shear_image(img, k):
    """Applies shearing to an image."""
    T = np.array([[1, k, 0],
                  [0, 1, 0]], dtype=np.float32)
    return cv2.warpAffine(img, T, (2*img.shape[1], 2*img.shape[0]))

def perspective_transform(img):
    """Applies perspective transformation."""
    h, w = img.shape[:2]
    src_pts = np.float32([[0, 0], [w-1, 0], [0, h-1], [w-1, h-1]])
    dst_pts = np.float32([[50, 50], [w-50, 0], [0, h-50], [w, h]])
    T = cv2.getPerspectiveTransform(src_pts, dst_pts)
    return cv2.warpPerspective(img, T, (w, h))

# Example translation
translated_img = translate_image(img, 100, 50)
show_image('Translated Image', translated_img)
cv2.imwrite('translated_img.jpg', translated_img)

# Example scaling
scaled_img_warp = scale_image_warp(img, 2.5, 2.5)
show_image("Scaled Image (warpAffine)", scaled_img_warp)
cv2.imwrite('scaled_img.jpg', scaled_img_warp)

# Show new dimensions
print("Original size:", img.shape)
print("Scaled size (warpAffine):", scaled_img_warp.shape)

# Example rotation
rotated_img = rotate_image(img, 10)
show_image("Rotated Image", rotated_img)
cv2.imwrite('rotated_img.jpg', rotated_img)

# Example shearing
sheared_img = shear_image(img, 0.3)
show_image("Sheared Image", sheared_img)
cv2.imwrite('sheared_img.jpg', sheared_img)

# Example perspective transformation
perspective_img = perspective_transform(img)
show_image("Perspective Transformed Image", perspective_img)
cv2.imwrite('perspective_img.jpg', perspective_img)

# Combined transformation
rotated_img = translate_image(rotate_image(img, 10), 40, -30)
show_image("Rotated Image", rotated_img)
cv2.imwrite('combined_transformed_img.jpg', rotated_img)

def create_transformation(angle, tx, ty, sx, sy, skew_factor, persp1, persp2):
    """Creates a 3x3 transformation matrix with rotation, translation, scaling, skew, and perspective."""

    # Rotation Matrix
    R = np.eye(3, dtype=np.float32)
    R[0, 0] = np.cos(angle)
    R[0, 1] = -np.sin(angle)
    R[1, 0] = np.sin(angle)
    R[1, 1] = np.cos(angle)

    # Translation Matrix
    t = np.eye(3, dtype=np.float32)
    t[0, 2] = tx
    t[1, 2] = ty

    # Scaling Matrix
    Scale = np.eye(3, dtype=np.float32)
    Scale[0, 0] = sx

```

```

Scale[1, 1] = sy

# Skew Matrix
Skew = np.eye(3, dtype=np.float32)
Skew[0, 1] = skew_factor # Shearing along x-axis

# Combined Transformation
T = t @ R @ Skew @ Scale # Matrix multiplication

# Perspective Transformation
T[2, 0] = persp1
T[2, 1] = persp2

return T

def apply_transformation(img, T, is_perspective):
    """Applies transformation to the image using matrix multiplication."""

    height, width = img.shape[:2]
    new_img = np.zeros_like(img)

    for i in range(height):
        for j in range(width):
            # Homogeneous coordinates
            p = np.array([[i], [j], [1]], dtype=np.float32)

            # Apply transformation
            new_p = T @ p
            new_z = 1.0

            if is_perspective: # Apply perspective division
                new_z = new_p[2, 0]

            new_x = int(new_p[0, 0] / new_z)
            new_y = int(new_p[1, 0] / new_z)

            # Check bounds before assigning pixels
            if 0 <= new_x < height and 0 <= new_y < width:
                new_img[new_x, new_y] = img[i, j]

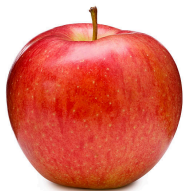
    return new_img

# Create transformation matrix
T = create_transformation(angle=0.81, tx=200, ty=150, sx=0.5, sy=0.5, skew_factor=0.2, persp1=0.0001, persp2=0.002)
print("T =", T)

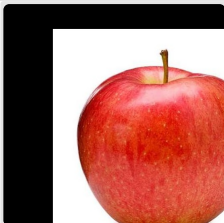
# Apply transformation
new_img = apply_transformation(img, T, is_perspective=True)
cv2.imwrite('apply_transformation.jpg', new_img)

show_image("Transformations", new_img)

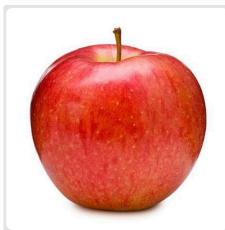
```



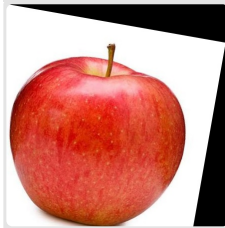
apple



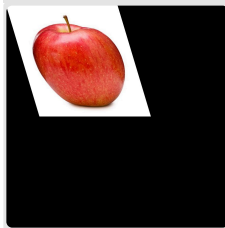
translated



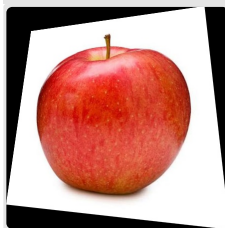
scaled



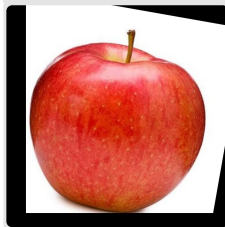
rotated



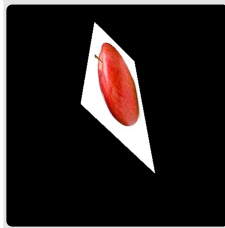
shread



perspective



combined_transform



apply_transform

Related:

1. Spatial Transformer network
2. Fully Convolutional Network (FCN)

References:

1. <https://chao-ji.github.io/jekyll/update/2018/07/19/BilinearResize.html>
2. <https://kevinzakka.github.io/2017/01/10/stn-part1/>

3. [3d-computer-vision](#). [3D-CV-Lab2](#), Tarlan Ahadi, Elte, 2025.